



Asignatura: Electiva Mecatrónica

**Tema a tratar: Investigación sobre los esquemáticos
que necesitamos para el proyecto**

Alumno: Nicairol Elían Medina Payano

Número de matrícula: 2021-2442

Facilitador/a: Carlos Antonio Pichardo Viuque

Fecha: 02/10/2025

Tarea 3-Investigación sobre los esquemáticos que necesitamos para el proyecto

1) ENTRADAS PNP — definición y diseño detallado

Qué es:

Sensor PNP (sourcing) que, en estado activo, conecta la salida a +V (normalmente +24 V). Cuando inactivo la salida queda en alta impedancia o 0 V según conexión.

Objetivo: llevar esa señal a un GPIO del ESP32 (3.3 V, entrada digital) con aislamiento y protección.

Circuito recomendado (seguro para ambiente industrial):

- Sensor PNP → **resistencia limitadora** → **LED de optoacoplador** → GND.
- Lado ESP32 del optoacoplador → salida colector-emisor, pull-up a 3.3 V y entrada al GPIO.
- Añadir TVS en entrada 24 V y un filtro RC si hay ruido.

Cálculo de resistor para el LED del opto (ejemplo):

Supongamos LED opto $V_f \approx 1.2$ V, queremos $I_f \approx 2$ mA, alimentación 24 V.

Cálculo paso a paso:

1. Tensión a disipar en resistor = $24.0 - 1.2 = 22.8$ V.
2. Resistencia $R = V / I = 22.8 \text{ V} / 0.002 \text{ A}$.
 - $22.8 \div 0.002 = 11400 \Omega$.
3. Elegir valor comercial: **12 kΩ** (o 11 kΩ / 10 kΩ con ajuste de corriente).

Protecciones y notas:

- Opto elimina bucles de masa.
- Añadir diodo de supresión (TVS) en 24 V y filtro RC (10 nF + 10 kΩ) si hay rebotes.
- Aislar la tierra de potencia y la tierra lógica lo más que el diseño permita (si no hay transformador, usar barrera de silicio).

Partes recomendadas: PC817, TLP521 (optos), TVS SMBJ24A.

2) SALIDAS PNP — definición y diseño detallado

Qué es:

Salida PNP es un elemento que **suministra +24 V** a la carga cuando se activa (sourcing). En tarjetas industriales: típicamente para alimentar válvulas, sensores pasivos, indicadores, etc.

Implementación con ESP32 (control desde 3.3 V):

- Usar un **driver de PNP** o invertir lógica con MOSFET/N-channel + driver P-channel si necesitas sourcing.
- Opción práctica: controlar un **rele o un transistor PNP/P-channel** que provea +24 V a la carga.

Circuito típico (sencillo y robusto):

ESP32 GPIO → driver MOSFET (N-channel) o transistor NPN → relé o circuito de conmutación que active un P-channel a +24 V (o usar driver de alto-lado como IRF9Z34 para P-channel).

Consideraciones:

- Si la carga es inductiva, colocar diodo flyback o supresión.
- Usar MOSFET con $R_{ds(on)}$ bajo si controlas cargas de corriente elevada.
- Si el sistema necesita “salida PNP” exactamente, usar driver dedicado de conmutación de alto lado (p. ej. MIC4427 para gates) o un **p-channel MOSFET** con gate protegido.

Partes sugeridas: AO3401 (P-channel SMD para baja potencia), IRLZ44 / Si2301 (según corriente).

3) ENTRADA 0–10 V — definición y diseño detallado

Qué es: señal analógica estándar (0–10 V) usada para control y medición (variadores, sensores).

Objetivo: leer 0–10 V con el ESP32 (ADC 0–3.3 V) sin perder precisión y con rechazo al ruido.

Soluciones:

1. **Divisor resistivo** de precisión + filtro RC + ADC externo (recomendado).
2. **Amplificador/condicionador** (op-amp) en configuración de atenuador con alta impedancia de entrada y protección de sobre-tensión.

Ejemplo práctico (divisor) — calcular resistencias para 10 V → 3.3 V máxima:

Queremos $V_{out} = V_{in} \times R_2 / (R_1 + R_2)$ y para $V_{in} = 10\text{ V}$, $V_{out} \leq 3.3\text{ V}$.

Ratio requerido = $3.3 / 10 = 0.33$.

Elegimos valores estándar: $R_1 = 21\text{ k}\Omega$, $R_2 = 10\text{ k}\Omega$.

Cálculo paso a paso:

1. $R_2 / (R_1 + R_2) = 10\text{ k} / (21\text{ k} + 10\text{ k}) = 10000 / 31000$.
2. $10000 \div 31000 = 0.32258\dots$
3. $V_{out} = 10\text{ V} \times 0.32258 = 3.2258\text{ V} \rightarrow$ **seguro** para ADC 3.3 V.

Mejoras: usar un **ADC externo** (ADS1115 16-bit I2C) para mayor resolución y medición diferencial. Añadir protección con diodos (clamp) y un fusible en línea.

Filtros: RC al voltaje atenuado (p. ej. $R = 10\text{ k}\Omega$ y $C = 4.7\text{ nF} \rightarrow f_c \approx 3.4\text{ kHz}$) para eliminar ruido de conmutación.

4) SALIDA 0–10 V — definición y diseño detallado

Qué es: generar una tensión analógica 0–10 V a partir del ESP32 (salida PWM o DAC a 3.3 V).

Opciones de implementación:

- **Opción A (más simple):** PWM del ESP32 → filtro pasa-bajo RC → **amplificador operacional** (buffer/gain) que convierta 0–3.3 V a 0–10 V.

- **Opción B (precisa):** DAC externo de 12–16 bits (p. ej. DAC80508 o MCP4728) + driver op-amp que eleve a 10 V y ofrezca baja impedancia.

Diseño ejemplo (op-amp con ganancia):

- Queremos ganar $10 / 3.3 \approx 3.03$.
- Usar op-amp rail-to-rail que soporte ± 12 V o alimentación dual, o alimentar op-amp con +12 V y GND si solo necesitas 0–10 V.

Notas de diseño:

- Usar op-amp con salida capaz de 0 V hasta casi su V_{cc} (rail-to-rail).
- Añadir filtro en la entrada (después del RC) para eliminar ripple de PWM.
- Para control fino, preferir DAC y amplificador de precisión.

Componentes sugeridos: MCP4725 (DAC I2C, requiere amplificador para 10 V), OPA376/OPA197 (op-amp rail-to-rail según fuente).

5) ENTRADA 4–20 mA — definición y diseño detallado

Qué es: estándar de lazo industrial donde 4–20 mA representa rango de medición (4 mA = mínimo, 20 mA = máximo). Ventaja: tolerante a ruido y longitudes de cable.

Conversión a tensión para ADC: usar una **resistencia de precisión (shunt)** para convertir corriente en voltaje.

Cálculo recomendado (ejemplo seguro para ADC de 3.3 V):

Queremos que $20 \text{ mA} \rightarrow \leq 3.0 \text{ V}$ (margen de 0.3 V con respecto a 3.3 V).

Usar $R = V / I = 3.0 \text{ V} / 0.02 \text{ A} = 150 \Omega$.

Cálculo paso a paso:

1. $20 \text{ mA} \times 150 \Omega = 0.020 \times 150 = 3.000 \text{ V}$.
2. $4 \text{ mA} \times 150 \Omega = 0.004 \times 150 = 0.600 \text{ V}$.

Resultado: rango 0.6 V–3.0 V, perfecto para ADC 0–3.3 V (p. ej. ADS1115). Usar **resistor de precisión 0.1% o 0.01%** y evitar grandes potencias ($P = I^2 R = 0.02^2 \times 150 = 0.0004 \times 150 = 0.06 \text{ W} \rightarrow 60 \text{ mW}$, por lo que 0.25 W es suficiente).

Alternativa profesional: usar un IC convertidor de corriente a voltaje o un amplificador de instrumentación (p. ej. XTR111 para bucle 4–20 mA).

Protecciones: serie con fusible, diodo antiparalelo, pararrayos/TVS para líneas largas.

6) SALIDA 4–20 mA — definición y diseño detallado

Qué es: generar una corriente de lazo 4–20 mA desde la tarjeta para controlar actuadores o transmisores que esperan lazo.

Opciones de implementación:

- **Driver dedicado (recomendado):** XTR115 / XTR116 / XTR111 (ICs diseñados para generar lazo 4–20 mA con DAC de referencia).

- **Implementación con DAC + op-amp current source:** DAC → op-amp + transistor para crear fuente de corriente proporcional.

Puntos de diseño:

- Precisión del DAC (12–16 bits).
- Referencia de voltaje estable (V_{ref} de 2.5 V o mejor).
- Aislamiento entre loop y MCU si necesario.

Protecciones: limitar máxima corriente, watchdog en firmware si falla la señal.

7) ENTRADAS NPN — definición y diseño detallado

Qué es: sensor NPN (sinking) pone la salida a **0 V** cuando se activa; común conectado a +24 V.

Objetivo: detectar el pulso (0 V activo) en GPIO 3.3 V.

Circuito típico (seguro):

Sensor NPN → optoacoplador o directo con resistencia y pull-up.

Preferible: **optoacoplador** para aislar; si no, usar divisor y protección.

Notas: invertir lógica en firmware (0 = activo). Añadir resistencias de pull-up (10–47 k Ω) a 3.3 V y filtrado de anti-rebote.

8) SALIDAS NPN — definición y diseño detallado

Qué es: salidas tipo NPN actúan como **colector abierto** (sinking): cuando activas conectan la carga al negativo.

Implementación:

ESP32 controla transistor N-channel o NPN que “lleva a masa” la carga. Excelente para relés y cargas industriales.

Circuito típico:

ESP32 → driver (MOSFET N-channel o transistor) → carga → +24 V. Añadir diodo flyback en cargas inductivas y TVS si necesario.

9) DRIVER RELÉ — definición y diseño detallado

Qué necesitas: controlar relés (mecánicos o SSR) desde ESP32 con protección y aislamiento.

Diseño típico:

- ESP32 GPIO → resistor de puerta/base → transistor (NPN o MOSFET) → relé coil → +Vcoil.
- **Diodo flyback** (1N4007) en paralelo con la bobina.
- Base/Gate resistor (10 k Ω a 100 Ω según transistor).
- Optoacoplador para aislamiento completo si es requerido.

Módulos prácticos: ULN2803A (8 canales con diodos internos) — ideal para múltiples relés.

Consideraciones de PCB: separar trazas de alta corriente, pads grandes para relés, usar condensadores cerca de la alimentación de la bobina.

10) ENTRADA DIGITAL (OPTOCOUPLER) — definición y diseño detallado

Por qué usar opto: aislar terreno de campo del microcontrolador, impedir daños por picos, romper bucles de masa.

Implementación típica: PC817 o TLP521.

- Lado campo: resistor limitador, LED interno del opto.
- Lado MCU: transistor fotocoupler con pull-up a 3.3 V y entrada al GPIO.

Detalles:

- Elegir opto con CTR suficiente para la corriente de LED disponible.
- Añadir filtro (R + C) para rebotes.
- Situaciones de alta velocidad: usar optos de alta velocidad o digital isolator (silicon) en lugar de opto.

11) INTERFAZ RS-485 — definición y diseño detallado

Qué es: estándar diferencial para comunicaciones seriales en ambientes ruidosos y largas distancias. Normalmente se usa con **Modbus RTU**.

Hardware mínimo:

- Transceiver RS-485 (MAX485, SN75176, MAX3485).
- Pines: DI (datos in), RO (datos out), DE/RE (Driver enable / Receiver enable) que se controlan con GPIO del ESP32 (combinar DE y RE si half-duplex).

Conexión física: A ↔ B (líneas diferenciales). Añadir **terminación 120 Ω** entre A y B en extremos. Añadir resistencias de polarización (fail-safe bias): pull-up en A y pull-down en B (valores típicos 560 Ω–1 kΩ) o usar resistencias de valor según datasheet del transceiver.

Protección en campo: TVS bidireccional en pair A/B, choke common-mode y supresión contra sobretensiones.

Control de DE/RE ejemplo:

- Para transmitir: set DE=HIGH, RE=HIGH, enviar datos y luego DE=LOW para recibir.
- Para recibir: DE=LOW, RE=LOW.

12) PROTOCOLO MODBUS (implementación en ESP32)

Qué es: protocolo maestro/esclavo serial usado ampliamente (Modbus RTU sobre RS-485).

Recomendaciones de implementación (firmware):

- Usar librerías probadas: ModbusRTU (Arduino/ESP32) o esp-modbus (en frameworks).
- Implementar **timeouts**, reintentos y CRC16.

- Manejar correctamente el control DE/RE para half-duplex RS-485 (no olvidar retrasos post-transmisión).
- Proveer diagnóstico y códigos de excepción en esclavos.

Consejos de diseño: soportar ser Master y Slave si el producto lo requiere; exponer registros de diagnóstico, entradas/outputs, y registros de configuración.

13) MEDICIÓN DE CT y AC (ICs recomendados)

Qué se mide: corriente y tensión AC con transformadores de corriente (CT) y transformadores/atenuadores para tensión. Objetivo: energía, potencia, factor de potencia, RMS.

Sensores:

- **CTs** (p. ej. SCT-013) para corriente no invasiva.
- **Shunts** o sensores Hall (ACS712/ACS758) para DC/AC (pero menos precisos que CTs para AC).

ICs para medición de energía (I2C preferible, SPI opcional):

- **ADE7953** — medidor de energía con varias funciones (puede usarse con I2C/SPI, chequear versión del paquete).
- **ADE7753 / ADE7758** — ICs de potencia (SPI típico).
- **ATM90E32** — muy usado en medidores modernos (SPI).
- **INA219** — mide corriente DC vía shunt (I2C) — no ideal para CT AC directa.
- **MCP3911 / MCP3903** — ADCs sigma-delta para mediciones de energía (SPI).

Ejemplo práctico de cadena de medición:

1. **CT** → burden resistor (transforma corriente a voltaje) → anti-aliasing filter → ADC/IC de energía.
2. **V sense** (transformador o divisor) → búfer/condicionador → ADC/IC de energía.
3. Realizar calibración en firmware (gain, offset), sincronización de muestreo y cálculo de RMS/potencia.

Diseño de burden resistor (ejemplo):

Si CT entrega 50 mA RMS a pico máximo y quieres 1 V RMS en burden: $R = 1 \text{ V} / 0.05 \text{ A} = 20 \text{ } \Omega$. Calcular potencia $P = I^2 R = 0.05^2 \times 20 = 0.05 \text{ W} \rightarrow 0.5 \text{ W}$? (hacer cálculo preciso en cada caso). Usa resistencia cerámica de potencia adecuada.

Notas: usar ICs con soporte de armónicos si necesitas THD o análisis de calidad de la energía.

PROTECCIONES GENERALES Y BUENAS PRÁCTICAS DE PCB

- **Aislamiento:** separar zona de potencia (24 V / RS-485) de la zona lógica; usar optos o transformadores.
- **TVS/Varistor:** en entradas de campo y en RS-485.
- **Condensadores de desacople:** 0.1 μF junto a cada IC y electrolítico bulk 10–47 μF en fuentes.

- **Trazado:** rutas de corriente alta separadas; estrella de GND donde la corriente de potencia regrese separadamente.
- **Separadores galvanicos si hace falta** (isolated DC-DC, digital isolators) para proteger MCU.
- **Test points** para calibración y medición.
- **Protección ante inversión de polaridad** en alimentaciones.

FIRMWARE — consideraciones clave

- **Lecturas ADC:** muestreo y promedio, filtrado digital (IIR), detección de saturación.
- **4–20 mA loop:** detectar condiciones de circuito abierto (<4 mA) y cortocircuito (>20 mA).
- **RS-485 / Modbus:** manejo de DE/RE, tiempos de retención post-TX, CRC y retries.
- **Watchdog y safe-state:** si MCU falla, dejar salidas en estado seguro.
- **Calibración:** parámetros de ganancia/offset guardables en EEPROM/Flash.
- **Protección:** clamping de entradas, límites y alarmas de firmware.

LISTA DE COMPONENTES RECOMENDADOS (resumen rápido)

- Optoacoplador digital: **PC817**, **TLP521**.
- ADC I2C: **ADS1115** (16-bit).
- DAC I2C: **MCP4725** (12-bit) o DAC de mayor resolución si necesitas.
- 4–20 mA driver: **XTR115/XTR116**.
- RS-485 transceiver: **MAX485**, **SN75176**, **SP3485**.
- Medición energía: **ADE7953**, **ADE7753**, **ATM90E32**, **MCP3911** (según necesidad).
- Driver relé: **ULN2803A** (múltiples canales) o transistores discretos (2N2222 / IRLZ44).
- MOSFET sugerido: **AO3400** (SMD N-channel), **AO3401** (P-channel).